# Hedge Funds on a Swamp: Analyzing Patterns, Vulnerabilities, and Defense Measures in Blockchain Bridges [Experiment, Analysis and Benchmark]

Poupak Azad
University of Manitoba
Canada
azad@myumanitoba.ca

Jiahua Xu
University College London
United Kingdom
jiahua.xu@ucl.ac.uk

Yebo Feng
Nanyang Technological University
Singapore
yebo.feng@ntu.edu.sg

Preston Strowbridge
University of Central Florida
USA
pr438045@ucf.edu

Cuneyt Gurcan Akcora
University of Central Florida
USA
cuneyt.akcora@ucf.edu

## SUPPLEMENTARY MATERIAL

## A RELATED WORK ON SCALABILITY, INTEROPERABILITY AND BRIDGE DESIGNS

Due to space constraints in the main text, this supplementary material outlines three additional research areas relevant to our work. We will start this section by analyzing scalability and interoperability studies.

### A.1 Blockchain Scalability and Off-Chain Protocols

Recent bridge proposals aim to shift trust off-chain and reduce exposure by using succinct proofs, validator diversity, or formal verification. The literature reflects an active effort to reconcile decentralization, trust minimization, and performance in cross-chain systems. For example, McCorry et al. [1] and Kim et al. [2] explore off-chain protocols and categorize scalability solutions, respectively. Blockchain interoperability, the ability for different blockchain systems to interact, is a core focus of Lee et al. [3], Wang [4], Belchior et al. [5], Zamyatin et al. [6], Qasse et al. [7], Monika et al. [8], Schulte et al. [9], Hardjono et al. [10], Lafourcade et al. [11], and Duan et al. [12]. These studies highlight the challenges in ensuring ACID properties across diverse blockchains [4], the limitations due to blockchain isolation [7], and the lack of interaction between different ledgers and legacy systems [8]. Methods vary: Lee et al. [3] focus on bridges, Zamyatin et al. [6] on communication protocols, Schulte et al. [9] suggest transitioning from closed to open systems, while Hardjono et al. [10] propose standardized architectures. Lafourcade et al. [11] advocate for a unified blockchain.

### A.2 Security and Privacy Concerns in Blockchain Interoperability

Security risks inherent to cross-chain protocols are discussed in Lee et al. [3], Zamyatin et al. [6], and Duan et al. [12]. Lee et al. examine bridge-specific vulnerabilities, Zamyatin et al. propose a trust-evaluation framework, and Duan et al. categorize systemic attack vectors. Borkowski et al. [13] provide a comprehensive review of cross-blockchain technologies through the TAST research project. Caldarelli [14] introduces wrapped tokens for interoperability, though with reintroduced trust assumptions. Borkowski et al. [15] explore atomic cross-chain transfers as a means to link otherwise isolated systems.

### A.3 Protocol Design, Relays, and Bridges

Pioneering work by Herlihy et al. [16] introduced the notion of cross-chain deals, formulating atomic cross-chain transactions under adversarial conditions. Their protocol ensures that mutually distrusting parties exchanging assets across separate blockchains either all succeed or all abort, without a trusted intermediary. Atomic cross-chain swaps and hashed timelock contracts emerged from this foundation [16], with further developments such as Anonymous Multi-Hop Locks [17] and Universal Atomic Swaps [18] generalizing these mechanisms for broader interoperability.

Relay and light-client protocols enable cross-chain state verification. BTC Relay provided a working, though costly, Ethereum-based verifier for Bitcoin transactions. Efficiency enhancements such as FlyClient [19] and NiPoPoW [20, 21] offer logarithmic communication overhead and succinct proofs. *zkBridge* demonstrates state transition verification via *zk-SNARKs* [22], showcasing low-cost, trust-minimized bridging across Ethereum and Cosmos.

Asset-based bridges also remain prominent. XCLAIM [23] introduced trustless, collateral-backed cross-chain assets. While projects like *Wormhole* and *PolyNetwork* use validator-based event notaries, they pose risks if quorum assumptions fail. These are complemented by formal frameworks such as *zkRelay* and *ETH Relay*, and conceptual models like Bitcontracts for Bitcoin–Ethereum smart contract interoperability.

Security of these bridges is increasingly scrutinized. SmartAxe [24] detects Cross-Chain Vulnerabilities through fine-grained static analysis. Attacks like the $320M *Wormhole* and $610M *PolyNetwork* incidents illustrate the cost of flawed validation and inadequate trust assumptions. Alba [21] proposes *Pay2Chain* bridges that combine off-chain payment guarantees with cryptographic proofs to reduce bridge attack surfaces.

## B PROOF OF THE FAILURE THEOREM

Theorem 1. *If a bridge experiences an attack or failure, then Equation* (**??**) *is always violated.*

**Table 1: Notations for Bridge Mechanisms**

| Notation | Description |
|---|---|
| $b$ | A blockchain. |
| $N_p$ | Set of peer-to-peer network nodes in a blockchain. |
| $N_a$ | Set of address nodes in a blockchain. |
| $\mathbb{H}$ | Chronological history of transactions in a blockchain. |
| $R$ | Consensus rules governing transaction creation in a blockchain. |
| $\theta$ | A token, with state defined by its transaction history $\mathbb{H}_\theta$. |
| $t$ | A point in time. |
| $tx$ | A transaction, defined as $tx = (\theta, v, a_1, a_2, t_{tx})$, transferring value $v$ of token $\theta$ from address $a_1$ to $a_2$ at time $t_{tx}$. |
| $a$ | A user address |
| $b_1, b_2$ | Source and destination blockchains in a bridge. |
| $\mathcal{I}$ | Implementation mechanism of a bridge. |
| $\mathbb{B}_{1\leftrightarrow 2}$ | A blockchain bridge between $b_1$ and $b_2$, defined as $\mathbb{B}_{1\leftrightarrow 2} = \{b_1, b_2, \mathcal{I}\}$. |
| $\chi t$ | The state of a cross-chain transaction at time $t$. |
| $TS$ | A node trust set |
| $c$ | A smart contract address |
| $v$ | Quantity of a token $\theta$. |
| $a \mapsto v$ | An address $a$ holds value $v$ of a token. |
| $F_{\text{forward}}$ | Bridge fee for forward transfer, defined as $F_{\text{forward}} = f_1 + f_2 + f^*$. |
| $F_{\text{reverse}}$ | Bridge fee for reverse transfer. |
| $f_1, f_2$ | Transaction processing fees on $b_1$ and $b_2$, respectively. |
| $f^*$ | Bridge operation fee. |
| $d$ | Duration associated with a blockchain operation |
| $\mathcal{T}_{\text{off}}$ | Off-chain Implementation |
| $d_{b_x}$ | Block confirmation durations for $b_x$. |
| $d^*$ | Time for off-chain mechanism to process and signal a transfer. |
| $\mathbb{T}$ | Bridge trust set. |
| $\mathbb{T}_{src}, \mathbb{T}_{dest}$ | Trust assumptions related to the source and destination blockchains. |
| $\mathbb{T}_{off}$ | Trust assumptions in off-chain mechanisms. |
| $L$ | Light client verifier. |
| $M$ | Merkle proof. |
| $N$ | Notary validator set. |
| $F$ | Total bridge fee for a transaction |
| $D$ | Delay introduced in a bridge due to proof. |
| $Q$ | Quantity of token being referenced. |
| $MP$ | Massage propagation |
| $V$ | Attack vector |
| $\mathbb{E}(C)$ | Expected loss or cost. |
| $\mathbb{E}(L)$ | Expected value at risk in a *CCB*. |
| $SCs$ | Set of smart contracts. |

Proof. Equation (??) formalizes token parity across two blockchains $b_1$ and $b_2$, requiring that the locked value on $b_1$ matches the released or minted value on $b_2$:

$$v_1 \cdot price(\theta_1, t) \equiv v_2 \cdot price(\theta_2, t), \quad \forall t.$$

Suppose a bridge undergoes an attack or failure. By Definitions ?? and ??, this implies that at least one of Equations (??), (??), or (??) is violated.

*Case 1: Violation of causality ((??)).* If a token transfer occurs on $b_2$ without a corresponding event on $b_1$, or vice versa, then either tokens are created without backing ($v_2 > v_1$), or locked value is unclaimed ($v_1 > v_2$). In both cases, the equivalence in (??) is broken.

*Case 2: Violation of consistency ((??) or (??)).* If a user holds tokens on $b_2$ while also being able to access funds on $b_1$, or if the one-to-one correspondence between locked and minted tokens is violated, then value duplication or loss occurs. Again, this leads to $v_1 \cdot price(\theta_1, t) \not\equiv v_2 \cdot price(\theta_2, t)$.

In all such cases, the bridge no longer preserves token parity, hence Equation (??) is violated.

□

# C CROSS-CHAIN BRIDGE SOLUTIONS

We will discuss several popular cross-chain bridge protocols, cross-referencing them with their formalization. Specifically, we examine i) the implementation choices made by each bridge at different layers, ii) the trust assumptions underlying each bridge, iii) fee and time considerations, including comments on security where relevant, iv) the specific blockchains that each bridge connects. Table 4 provides an overview of cross-chain bridge implementations and architectural components.

## C.1 Arbitrum Bridge

Arbitrum is not a bridge itself, but a layer two rollup solution designed to prepare large batches of transactions off-chain, significantly reducing gas costs and scaling Ethereum. However, Arbitrum also includes a native token bridge, which lets users move assets between Ethereum (L1) and Arbitrum (L2). This is commonly referred to as the Arbitrum Bridge, and it's used to deposit and withdraw tokens like ETH or ERC-20s.

As an optimistic rollup, Arbitrum assumes transaction validity by default and relies on economic incentives to detect and correct fraud. A centralized sequencer currently assembles and orders transaction batches, producing a new Arbitrum block for each Ethereum block. These transactions are compressed and posted as calldata (read-only, byte-addressable region) on Ethereum.

A set of smart contracts deployed on Ethereum manages the core protocol logic, including the rollup mechanism, cross-chain messaging, fraud proofs, bridging, and third-party gateway interactions. The trust set is defined as $\tau = \{\text{SCs}\}$, representing these deployed smart contracts, which collectively enforce the security and correctness of the rollup.

*Arbitrum's bridge* is structured around a gateway bridge contract that delegates to more specific contracts (e.g., ERC20, WETH, or custom contracts). These specialized contracts are responsible for minting the corresponding tokens on layer 2. This modular design

enhances security and extensibility by isolating token-specific logic across separate contracts.

To guard against fraud, Arbitrum implements a challenge-response protocol during a designated challenge period. Off-chain participants can dispute the validity of a rollup block through an interactive verification process often described as the "grasshopper game", where the correct and fraudulent segments of execution are iteratively narrowed down. If fraud is detected, the protocol rolls back to the last valid state and removes the invalid block. A successful challenger receives a financial reward, typically derived from the stake of the fraudulent party. Conversely, false challengers lose their stake, discouraging frivolous disputes.

Finality on Arbitrum is delayed by the challenge period, which affects asset bridging. Users must wait until this period ends before safely withdrawing assets to Ethereum. Although this delay is inherent to optimistic security models, it is a practical consideration in bridge comparisons.

## C.2 Wormhole protocol

*Wormhole* defines itself as a cross-chain message-passing protocol that enables communication between heterogeneous blockchains. Bridging protocols are built on top of *Wormhole* to facilitate token and data transfers across chains.

On-chain, *Wormhole* uses a set of smart contracts deployed to both the source and destination chains. The emitter contract on the source chain initiates a transfer by invoking the *'publishMessage'* method on the *Wormhole* core contract. These core contracts are responsible for emitting and verifying cross-chain messages, forming the trust set $\tau = \{\text{SCs}\}$ on both ends. The user relies on the correct and secure execution of these smart contracts for the protocol's integrity.

The off-chain implementation $\mathcal{T}_{off}$ relies heavily on a guardian network, composed of 19 independent nodes operated by various institutional entities. These guardians observe on-chain activity, verify messages, and collectively sign a Verifiable Action Approval (VAA). A quorum of at least 13 of 19 guardians must sign the VAA for it to be considered valid. This introduces an additional trust assumption, so the full trust set includes off-chain actors, i.e., $\tau \neq \{\}$. In practice, this makes *Wormhole* a trusted bridge, as its security hinges on the honest majority of the guardian set.

Once the VAA is generated, a relayer (which may be a user or a third party) delivers the signed message to the destination chain. There, the *Wormhole* core contract verifies the guardian signatures and executes the associated instructions via the target smart contract. As with the source chain, the destination chain also has a trust set $\tau = \{\text{SCs}\}$.

Fee and time considerations in *Wormhole* are variable. Since the relayer role is permissionless, users or third parties may bear the cost of relaying messages across chains, and some applications built on *Wormhole* subsidize or automate these steps. Finality and latency depend on the source chain's block confirmation time, guardian network processing, and delivery to the destination chain. As such, it does not provide instant finality but is generally faster than optimistic rollups that require challenge periods.

*Wormhole* connects a broad range of blockchains, including Solana, Algorand, Near [25], Aptos [26], Sui [27], and CosmWasm chains [28], as well as EVM-compatible blockchains such as Ethereum, Arbitrum, Avalanche, Polygon, Base [29], and Optimism. This makes it one of the most widely integrated cross-chain messaging protocols.

## C.3 Celer *cBridge*

Celer *cBridge* offers two models for cross-chain asset transfers: a liquidity-pool-based model and an OpenCanonical token bridging model. In the pool-based model, liquidity providers lock assets into pools on both the source and destination chains, allowing users to swap assets between chains directly. In the OpenCanonical model, token transfers follow a mint-and-burn approach. On the source chain, tokens are locked in a smart contract called *TokenVault*, initiating the bridging process. The trust set on the source chain is $\tau = \{\text{SCs}\}$, as users must rely on the correct execution of these smart contracts.

The off-chain implementation $\mathcal{T}_{\text{off}}$ involves the State Guardian Network (SGN), a Cosmos-based proof-of-stake blockchain built on Tendermint. SGN acts as a validator and orchestrator of the bridge, confirming transactions off-chain. Since SGN is a third blockchain mediating between the source and destination chains, *cBridge* operates as a sidechain-based bridge. The system's trust assumptions, therefore, include the consensus protocol of SGN, denoted $\mathfrak{R}_{\text{SC}}$, as well as the live correctness of validators (*LC*), message propagation (*MP*), and the honest majority of SGN nodes (*N*). The resulting trust set is $\tau = (\{LC, MP\} \cap \{N\}) \cup \mathfrak{R}_{\text{SC}}$, characterizing *cBridge* as a trust-minimized bridge relative to fully trusted notary-based models.

After validation by SGN, the transaction is relayed to the *PeggedToken* contract on the destination chain, which mints pegged tokens to complete the transfer. The destination chain's trust set is again $\tau = \{\text{SCs}\}$, as users rely on the correct behavior of the deployed smart contracts.

An additional layer of optional security is available through an optimistic-rollup-style model. In this mode, before SGN finalizes a transfer, there is a buffer delay period during which transactions can be independently verified. If inconsistencies are detected, the transfer can be halted, offering an added layer of fraud prevention.

Fee and time considerations for *cBridge* vary by model and configuration. Both *xAsset* (mint-and-burn) and *xLiquidity* (pool-based) bridges impose a base fee and a protocol fee. The base fee includes the destination chain's gas costs, while the protocol fee, which ranges from 0 percent to 2 percent, depends on the source and destination chains and the transfer amount. This fee compensates SGN validators and stakers for securing the protocol.

*cBridge* currently supports a wide range of chains (41 at the time of this writing), including Ethereum, Arbitrum, Avalanche, BNB Chain, Celo [30], Polygon, and Fantom, among others.

## C.4 Avalanche Bridge

The *Avalanche Bridge* is designed to enable fast, secure, and low-cost asset transfers between Avalanche and external networks such as Ethereum and Bitcoin. On the source chain, a smart contract is invoked to initiate the bridging process, forming a trust set $\tau = \{\text{SCs}\}$, where users rely on the correct execution of the deployed smart contracts.

The off-chain implementation $\mathcal{T}_{\text{off}}$ of the *Avalanche Bridge* is distinctive and combines a committee-based relayer design with hardware-based verification. A group of eight relayers, known as Wardens, monitor on-chain events and submit signed messages to a secure enclave powered by Intel SGX. This enclave, running verified and attested code, acts as a trusted execution environment that verifies the correctness of submitted data. Once at least six out of eight Wardens agree on the same transaction, the SGX enclave signs off on the transfer. This architecture introduces off-chain trust assumptions, and the trust set satisfies $\tau \neq \{\}$, making *Avalanche Bridge* a trusted bridge. Although the Warden set is geographically and institutionally distributed, the need for a quorum and reliance on Intel SGX introduces centralization risks in the event of collusion or compromise.

On the destination chain, the implementation $\mathcal{T}_{\text{dest}}$ involves a validator-controlled process. After off-chain validation, a smart contract is called to mint the bridged tokens. While this minting process is smart contract-based, finality is dependent on the decentralized consensus of the underlying blockchain (e.g., Avalanche C-Chain), where validators validate the resulting state. As this process is fully decentralized, the trust set at this stage can be written as $\tau = \{\}$.

Fee and time considerations vary depending on the asset and direction of transfer. Transfers from Ethereum to Avalanche typically take around 15–20 minutes, while Bitcoin transfers may take up to 1 hour due to Bitcoin's block time. On the Avalanche side, confirmations complete within seconds. ERC-20 tokens sent from Ethereum to Avalanche incur a 0.025 percent fee (minimum USD 3, maximum USD 250), while USDC is charged at a reduced rate of 0.02 percent (minimum USD 3, maximum USD 250). Bridging WBTC or WETH from Ethereum to Avalanche incurs a flat fee of USD 3. Transfers in the opposite direction, from Avalanche to Ethereum, are charged a 0.1 percent fee (minimum USD 12, maximum USD 1000). WBTC and WETH offboarding carries a flat USD 20 fee. Native Bitcoin transfers to Avalanche incur a minimum fee of approximately USD 3 in BTC and result in the minting of BTC.b tokens. Returning BTC.b from Avalanche to the Bitcoin network incurs a fee of approximately USD 20 plus Bitcoin miner fees, with a minimum transfer threshold in place to ensure the transaction is economically viable.

The *Avalanche Bridge* currently supports bi-directional cross-chain transfers between Ethereum or Bitcoin and the Avalanche C-Chain. ERC-20 tokens bridged from Ethereum are wrapped and appear with a `.e` suffix (e.g., `USDC.e`, `WBTC.e`), while Bitcoin is represented as `BTC.b`. One exception is USDC, which now uses Circle's Cross-Chain Transfer Protocol (CCTP) and is natively burned and minted on each side, removing the `.e` suffix. Avalanche-native assets cannot be bridged to other chains using the Avalanche Bridge.

## C.5 Multichain

*Multichain*, formerly known as Anyswap, presents itself as infrastructure for arbitrary cross-chain interactions. On the source chain, the bridge locks the user's assets in a smart contract that operates under a Secure Multi-Party Computation (SMPC) scheme. This smart contract is referred to by *Multichain* as a Decentralized Management Account, which securely holds assets during the bridging process. The source chain trust set is $\tau = \{\text{SCs}\}$, as users must trust the correct behavior of this contract.

The off-chain implementation $\mathcal{T}_{\text{off}}$ relies on a network of SMPC nodes, which collectively validate and sign transactions before initiating actions on the destination chain. These nodes serve as notaries in the bridging process, making the trust set $\tau \neq \{\}$, and thus *Multichain* is categorized as a trusted bridge. Because the off-chain signing controls minting and releasing assets, the integrity of this notary network is critical to the bridge's security.

On the destination chain, once a transaction is validated off-chain by the SMPC network, a smart contract responsible for minting wrapped assets is triggered. The tokens are minted 1:1 relative to the assets locked in the Decentralized Management Account on the source chain. The trust set on the destination chain is again $\tau = \{\text{SCs}\}$, reflecting reliance on the execution of deployed smart contracts. For reverse transfers, the wrapped token contract burns tokens on the destination chain, and the SMPC nodes authorize the release of the original assets from the source chain's smart contract.

Fee and time considerations depend on the chain and transaction size. For non-Ethereum chains, the cross-chain fee ranges from USD 0.9 to USD 1.9. For Ethereum, a 0.1 percent fee applies, with a minimum of USD 40 and a maximum of USD 1000. Minimum transfer amounts are USD 12 for non-Ethereum destinations and USD 50 when bridging to Ethereum. Maximum transfer size is capped at USD 20 million, though transactions above USD 5 million may incur delays of up to 12 hours.

*Multichain* supports a broad range of blockchains, including Bitcoin, Litecoin [31], Blocknet [32], ColossusXT [33], Terra [34], and many EVM-compatible networks such as Ethereum, Avalanche C-Chain, Binance Smart Chain, Celo, Polygon, and Arbitrum. This wide integration makes *Multichain* one of the most expansive cross-chain bridges currently available.

**Table 2: Connectivity matrix of major blockchain networks based on existing bridge protocols. Each cell indicates whether a direct bridge exists and, if so, categorizes the bridge by its trust model: TL: Trustless. Verification relies solely on the blockchains' own consensus (no third-party custodian), TM: Trust-Minimized. Decentralized validators or bonded operators secure the bridge (additional assumptions, but no single custodian), and TR: Trusted. A custodial or centralized entity/committee secures the bridge (users must fully trust this intermediary).**

| Chains | BTC | ETH | SOL | AVAX | ATOM | NEAR | DOT | XRP | XLM | ALGO | ADA | ARB | OP | MATIC | BNB |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **BTC** | – | TM | TM | TM | TM | TL | TL | TM | | TR | TL | TL | TM | TM | TM |
| **ETH** | TM | – | TM | TM | TM | TL | TL | TM | TM | TM | TM | TL | TL | TR | TM |
| **SOL** | TM | TM | – | TM | | TM | | TM | TM | TM | | TM | TM | TM | TM |
| **AVAX** | TM | TM | TM | – | TM | TM | | TM | TM | TM | TM | TM | TM | TM | TM |
| **ATOM** | TM | TM | | TM | – | | TL | | | | | TM | TM | TM | TM |
| **NEAR** | TL | TL | TM | TM | | – | | | | | | TM | TM | TM | TM |
| **DOT** | TL | TL | | | TL | | – | | TM | | | TM | TM | TM | TM |
| **XRP** | TM | TM | TM | TM | | | | – | TM | | TM | TM | TM | TM | TM |
| **XLM** | | TM | TM | TM | | | TM | TM | – | | | | | TM | TM |
| **ALGO** | TR | TM | TM | TM | | | | | | – | | TM | TM | TM | TM |
| **ADA** | TL | TM | | TM | | | | TM | | | – | TM | TM | TM | TM |
| **ARB** | TL | TL | TM | TM | TM | TM | TM | TM | | TM | TM | – | TM | TM | TM |
| **OP** | TM | TL | TM | TM | TM | TM | TM | TM | | TM | TM | TM | – | TM | TM |
| **MATIC** | TM | TR | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | – | TM |
| **BNB** | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | TM | – |

Table 2 shows a connectivity matrix for major blockchains and selected Layer-2 networks. Each cell is color-coded to indicate the

nature of the bridge connection (if one exists) between the row and column blockchains: Trustless, Trust-Minimized, or Trusted. For example, *Rainbow Bridge* connecting Ethereum and NEAR is trustless, while *Binance Bridge* (BNB Chain) and *Polygon PoS* Bridge are trusted/custodial solutions. *Cosmos IBC* and Polkadot's upcoming *Snowbridge* are also designed to be trustless. In contrast, multisignature or externally validated bridges like *Wormhole* and *Multichain* are trust-minimized (they decentralize validation but still rely on additional trust assumptions beyond the base blockchains. Blank cells indicate no widely used direct bridge between those networks. All entries are based on up-to-date bridge protocols (e.g., *Wormhole*, *Multichain*, *LayerZero*, *IBC*, etc.) and public documentation of their trust models.

# D STATE-OF-ART FOR BLOCKCHAIN BRIDGES

Blockchain interoperability has created an ecosystem of bridges linking most L1 and L2 networks. For example, the Wormhole protocol alone connects 23+ blockchains across six different smart contract runtimes [35], and the Axelar network bridges over 40 chains [36]. In parallel, Ethereum serves as a hub in this web; many chains deploy bridges to Ethereum to tap its liquidity and user base. Even previously siloed platforms are integrating. For instance, Cardano [37] has explored the Inter-Blockchain Communication (IBC) Protocol [38] to join the Cosmos network.

## D.1 Trust Models

Classic examples of trustless bridges fall into two categories: L1-to-L1 bridges, such as IBC, and L1-to-L2 bridges, such as in Arbitrum. On IBC channels, light clients on each chain validate each other's consensus state [39], so IBC's security reduces to the underlying chains' security with no additional trusted parties (native chain security). Rollup-based L1-to-L2 bridges fall into optimistic and zero-knowledge-based bridges [40]. A zero-knowledge rollup bridge includes a validity proof with every state update, so Ethereum only accepts withdrawals with a valid proof, whereas an optimistic rollup bridge relies on fraud proofs with a challenge period. In optimistic rollup bridges, the bridge inherits the source chain's security and does not introduce new trust assumptions, aside from at least one honest watcher in optimistic systems [41]. Indeed, industry analyses consider Ethereum's rollup bridges (L1-to-L2) among the most trustless interoperability solutions available [42].

Rollups cannot facilitate bridging between L1 chains with different runtimes (e.g., Bitcoin and Ethereum). As a result, most L1-to-L1 connectivity relies on trust-minimized intermediaries (i.e., networks of validators, relays, or oracles) that verify transactions across chains. Users must trust that a majority of these actors behave honestly, often incentivized through mechanisms like staked collateral or slashing penalties. Examples include *Axelar* [43], *Wormhole* [44], *LayerZero* [45], *Multichain* [46], and Celer's *cBridge* [47]. These permissioned or federated bridges require a predefined set of entities to authorize transfers and offer stronger assurances than single custodians, but fall short of the security provided by native chain verification. For instance, bridging assets from Ethereum to *BNB Chain* [48] or Solana often involves a validator coalition verifying the source chain deposit and minting a wrapped asset on the destination chain.

At the far end of the spectrum are trusted bridges, which include custodial bridges (e.g., wrapped Bitcoin via *BitGo*'s WBTC, where a single custodian holds the asset reserve) and some early L1-to-L1 bridges run by a single team or exchange. A notable example was Binance's original bridge connecting Ethereum and *BSC* (*BNB Smart Chain*) [49], essentially operated by Binance as a trusted custodian. Similarly, the *Ronin bridge* for Axie Infinity (prior to its 2022 hack) relied on just nine validators controlled by a single organization (Sky Mavis) and its partners, effectively a federated multisig under one entity's control. Such trusted models have minimal decentralization: users must trust that the custodian or signers will never misbehave, as there are no protocol-level penalties or verifications of their actions. While simple and fast to deploy, these bridges have the weakest security guarantees and have often proven fragile under attack, such as the *Ronin bridge* hack of 2022 (see Table 3). Even when the underlying bridge technology is secure, adversaries may compromise physical infrastructure to exfiltrate private keys.

**Table 3: Documented security incidents and failures in cross-chain bridge and routing architectures. Most attacks involve violations of the cross-chain causality prior.**

| Bridge | Architecture Type | Date (↓) | Loss (USD) | Technique | Violation | Target | Vector |
|---|---|---|---|---|---|---|---|
| Thorchain | Sidechain/Relay | 2021-06-29 | 140K | False top-up | Causality | Source chain | V3 |
| ChainSwap | Notary | 2021-07-02 | 800K | Contract vulnerability | Consistency | Source chain | V3 |
| ChainSwap 2 | Notary | 2021-07-11 | 4M | Contract vulnerability | Consistency | Source chain | V3 |
| Multichain | MPC (custodial) | 2021-07-10 | 7.9M | Private key compromised (bad ECDSA) | Causality | Source chain | V13 |
| Thorchain 2 | Sidechain/Relay | 2021-07-15 | 7.6M | False top-up | Causality | Source chain | V9 |
| Thorchain 3 | Sidechain/Relay | 2021-07-22 | 8M | Refund logic exploit | Causality | Source chain | V3 |
| Thorchain 4 | Sidechain/Relay | 2021-07-24 | 76K | Phishing attack | Causality | Off-chain relayer | V23 |
| Poly Network | Notary | 2021-08-10 | 611M | Trusted state root exploit | Causality | Source chain | V9 |
| pNetwork | Notary | 2021-09-20 | 13M | Inconsistent event parsing | Consistency | Off-chain relayer | V6 |
| Plasma Bridge | Sidechain/Relay | 2021-10-05 | None | Reused burn proof | Consistency | Destination chain | V15 |
| Optics Bridge | Notary | 2021-11-23 | None | Multi-signature permission vulnerability | Causality | Off-chain relayer | V3 |
| Multichain 2 | MPC (custodial) | 2022-01-18 | 1.4M | Token contract vulnerability | Consistency | Source chain | V3 |
| Qubit Finance | Relay | 2022-01-28 | 80M | Deposit function exploit | Causality | Source chain | V3 |
| Wormhole | Validator-based | 2022-02-02 | 326M | Signature exploit | Causality | Destination chain | V12 |
| Meter | Sidechain/Relay | 2022-02-06 | 7.7M | Deposit function exploit | Causality | Source chain | V3 |
| Ronin | Sidechain | 2022-03-23 | 624M | Private key compromised (social engineering) | Causality | Off-chain relayer | V13 |
| Marvin Inu | Custodial | 2022-04-11 | 350K | Private key compromise | Causality | Source chain | V13 |
| QANX Bridge | Notary | 2022-05-18 | 2.2M | Deploy fake bridge contract onto BSC | Consistency | Source chain | V3 |
| Horizon Bridge | Notary | 2022-06-23 | 100M | Private key compromised (unknown method) | Causality | Off-chain relayer | V12 |
| Nomad | Relay | 2022-08-01 | 190M | Trusted state root exploit | Causality | Destination chain | V10 |
| Celer | Sidechain/Relay | 2022-08-18 | 240K | DNS cache poisoning attack on frontend UI appprox | Causality | Off-chain relayer | V23 |
| Omni Bridge | Notary | 2022-09-18 | 290K | Possible ChainID vulnerability | Consistency | Destination chain | V15 |
| Binance Bridge | Custodial | 2022-10-06 | 570M | Proof Verifier Bug | Consistency | Off-chain relayer | V9 |
| QANX Bridge 2 | Notary | 2022-10-11 | 2M | Weak address key vulnerability | Consistency | Destination chain | V13 |
| Thorchain 5 | Sidechain/Relay | 2022-10-28 | None | Network interruption | Causality | Off-chain relayer | V17 |
| pNetwork | Hash-locking | 2022-11-04 | 10.8M | Misconfiguration of the pNetwork-powered bridge | Consistency | Token contract | V3 |
| Multichain 3 | MPC (custodial) | 2023-02-15 | 130K | Rush attack | Consistency | Source chain | V13 |
| Dexible | Trade router | 2023-02-20 | 2M | Unchecked destination address | Consistency | Source chain | V5 |
| Allbridge | Sidechain/Relay | 2023-04-02 | 570K | Logic flaw in withdraw function | Consistency | Destination chain | V2 |
| Celer | Hybrid | 2023-05-24 | None | Double-voting vulnerability | Causality | Off-chain relayer | V18 |
| Poly Network | Notary | 2023-07-02 | 10M | Private key compromised | Causality | Off-chain relayer | V13 |
| Poly Network | Notary | 2023-07-03 | 4.4M | Compromised multisig | Causality | Off-chain component | V13 |
| Multichain 4 | MPC (custodial) | 2023-07-07 | 126M | Compromised address | Causality | Off-chain component | V13 |
| Multichain 5 | MPC (custodial) | 2023-07-14 | unknown | State seized multisigs | Causality | Off-chain component | V12 |
| Shibarium | L2 Sequencer Bridge | 2023-08-17 | None | Frozen withdrawals due to L2 bug | Causality | Off-chain component | V17 |
| HTX | MPC (custodial) | 2023-11-22 | 12.5M | Hot wallet compromised | Causality | Off-chain component | V13 |
| HECO Bridge | MPC (custodial) | 2023-11-22 | 86.6M | Hot wallet compromised | Causality | Off-chain component | V13 |
| HYPR Network | Sidechain/Relay | 2023-12-13 | 220K | Vulnerability in external code dependency | Consistency | Destination chain | V7 |
| Socket's Bungee | Aggregator (router) | 2024-01-16 | 3.3M | Compromised multisig | Consistency | Source chain | V12 |
| Alex | Notary | 2024-05-14 | 4.3M | Compromised deployer | Causality | Source chain | V12, V7 |
| Lifi/Jumper | Aggregator (meta-router) | 2024-07-16 | 9.73M | Buggy function | Consistency | Source chain | V2 |
| Ronin 2 | Sidechain | 2024-08-06 | 12M | Whitehat MEV attack (parameter error in update) | Causality | Source chain | V4 |
| Feed Every Gorilla | Notary (Wormhole-based) | 2024-12-30 | 1.3M | Message spoofing | Causality | Source chain | V9 |

## D.2 Connectivity Trends

Supplementary material Table 2 shows the current bridge topology with clear patterns. Ecosystems with homogeneous technology (e.g., shared virtual machine) use trustless native bridges among themselves, whereas heterogeneous connections (e.g., Bitcoin-Ethereum) gravitate toward trust-minimized hubs. The Cosmos ecosystem is a prime example of the former: dozens of Cosmos-SDK chains (e.g., Osmosis [50], Cosmos Hub, Cronos [51], and Juno [51]) all interconnect via IBC channels with no external mediators.

To bridge out to Ethereum and other ecosystems, Cosmos projects deployed separate bridge modules and networks. For example, the *Gravity Bridge chain* [52] and Axelar network act as Cosmos-Ethereum L1-to-L1 bridges by having their validator sets observe

events on Ethereum and vice versa [53]. These are trust-minimized solutions (essentially multisig validator bridges) added onto Cosmos to import assets like USDC, DAI, and WETH into the IBC realm.

Ethereum and its orbiting chains illustrate a different connectivity paradigm. Virtually every major L1 (Solana, BNB Chain, Avalanche, Polygon, Fantom [54], etc.) has at least one bridge to Ethereum, given the high value of assets and liquidity on Ethereum. For example, *Wormhole* operates as a cross-chain message network connecting Ethereum with Solana, BSC, Avalanche, Polygon, and others, using a guardian consensus (a set of 19 nodes) to validate transfers. When a user moves ETH from Ethereum to Solana via the L1-to-L1 *Wormhole*, the ETH is locked in a *Wormhole* contract on Ethereum, and a wrapped ETH is minted on Solana, based on a signed attestation by the guardian network. This design is trust-minimized (no single custodian, but users trust the majority of guardians).

Other popular Ethereum bridges follow similar patterns: *Multichain* (Anyswap) uses a rotated set of Multi Party Computation signers to custody and mint assets between Ethereum and EVM chains; Celer *cBridge* uses a Proof-of-Stake validator set to oversee transfers. In all these cases, Ethereum-to-L1 bridges tend to introduce a separate middleware layer.

Outside the Ethereum/Cosmos spheres, other ecosystems have pursued their own bridging approaches with mixed strategies. Solana, for instance, developed ties to Ethereum and others mainly through third-party bridge networks. *Wormhole*'s origin was as the Solana-Ethereum L1-to-L1 bridge (also known as *Portal*), and it remains the primary connector between Solana and multiple other chains.

One constraint Solana faced is that its very high throughput (2.9K - 65K tx/sec) and non-EVM-based execution environment meant fewer independent bridge options. The ecosystem largely relied on *Wormhole*, which introduced a single point of failure for connectivity. Recent projects are working on light-client bridges (e.g., using Solana's clients), and even Cosmos's IBC was adapted in 2023 to work from Solana via an adapter chain. However, Solana's cross-chain connectivity is primarily driven by multi-signature bridge protocols rather than native, trustless channels. The BNB Chain ecosystem likewise began with a highly centralized bridging model. BNB Chain comprises a dual-chain system, consisting of an application-focused Beacon Chain and an EVM-compatible Smart Chain, bridged by the "Token Hub". The BSC Token Hub initially operated under the tight control of a few validators, as evidenced by the fact that a successful attack in October 6, 2022 was able to forge just two messages and mint 2 million BNB (see Table 3, an exploit that even forced a temporary halt to the chain. In response, BNB Chain has moved to increase the security of its bridges (e.g., raising multisig thresholds and improving key management) and has also encouraged the use of external bridges.

Nowadays, BNB Chain is connected to Ethereum and others via both Binance's official L1-to-L1 bridge (still centrally governed) and alternatives like *Celer*, *deBridge* [55], and *LayerZero* [45], which are more decentralized. Nevertheless, BNB's connectivity strategy remains cautious: its native bridge prioritizes speed and user experience (at the cost of trust), whereas third-party solutions offer decentralization but introduce dependency on external validator sets. We observe a similar pattern in other ecosystems, such as Polygon (the *Polygon PoS* bridge utilizes a 5-of-8 multisig to checkpoint withdrawals, a somewhat trusted design, alongside newer trust-minimized options) and Tron (which relies on custodial bridges for assets like BTC/ETH via BitGo, or exchange-mediated transfers). In contrast, Polkadot [56] takes a route closer to Cosmos: all parachains in a Polkadot parachain cluster are natively interoperable via the XCM (Cross-Consensus Message) format [57], which is "trustless and secure" for in-network messaging. Polkadot's relay chain ensures the validity of messages between parachains, so within this ecosystem, cross-chain actions (such as token transfers and remote contract calls) don't require external trust. However, bridging Polkadot to external chains still entails separate bridge projects (often with their own validator sets or light clients). Efforts are underway (e.g., Snowfork and Darwinia for Ethereum-Polkadot L1-to-L1 bridges) to make those as trust-minimized as possible, potentially even integrating light-client verification.

## E DESCRIPTIONS OF †ATTACK VECTORS

In Table **??**, we used † to indicate that these vulnerabilities only apply under certain design conditions. We now elaborate on those conditions.

**V1: Fake Burn/Lock Proofs.** Tokens are locked or burned on the source chain, X, before minting can occur on the destination chain, Y. A cryptographic proof sent from X to Y is required for confirmation before minting can occur. This attack involves forging the proof somehow to trick the Y chain into minting before any proper burn or lock has actually happened.

**V2: Malicious Transaction Modification.** Transaction details (addresses, number of tokens, gas fees, etc.) are sent from the source chain to the destination chain during a transaction. Malicious transaction modification involves somehow intercepting this detailed payload and asserting some fake payload. This can be done off-chain with protocols that include some centralized sequencer, or on-chain when the destination bridge is looking for the detailed payload.

**V3: Reentry Attack** Occurs when some external function is called multiple times before the initial execution has completed. This attack vector was used in the famous DAO attack, where the token minting was executed many times before the balance was checked and updated.

**V4: Integer Overflow/Underflow** When overflow or underflow remain unchecked, attackers can induce attacks by creating unexpected results. A wrap-around can also happen, where the maximum value can wrap around to the minimum value, or vice versa, creating room for exploitation.

**V5: Access Control Flaws** Access control is very important in maintaining control over a contract, and if unchecked, an attacker can impersonate a contract owner and manipulate the entire contract.

**V6: Timestamp Manipulation** A large number of blockchain protocols require some timestamp with loose limits to allow for network delays. These loose limits allow attackers to falsify timestamps that will still be considered valid by the protocol, leading to vulnerable behavior in the protocol.

**V7: Inconsistent Cross-Chain Transfers** Bridges can implement different methods for withdrawing and depositing, such as burning when performing a withdrawal but minting on a deposit. When these operations are different, attackers can take advantage of a lack of one-to-one consistency.
**V8: Replay Attacks** Attackers can take a message in transit that was validated, then delay and retransmit the original message in order to assume validation but cause effects unintended by the original message.
**V9: Oracle Manipulation** Oracles being used to provide data, such as token pricing, can be points of vulnerability if an attacker were to take control of the oracle and provide fake data, leading to incorrect fees or token amounts during minting or burning. Oracles can provide more data than just token prices, leading to more points of possible attack.
**V10: Consensus Failure** Commonly referred to as a 51% attack, if an attacker were to control 51% of the mining or staking power (depending on the protocol) then that attacker would be able to create a new chain segment that builds faster than the legitimate chain. This control can lead to false transactions that could mint tokens on other chains.
**V11: Malicious Custodian Manipulation** Custodial wallets are commonly used to hold locked tokens or private keys via a centralized source. Compromising or maliciously colluding with this custodial wallet leads to access of locked tokens, private keys, or other sensitive data.
**V12: Key Leakage / Private Key Theft** Locked assets can exist within bridges accessible by private keys and bridges themselves can hold keys along with their controlling proxy contracts. Any compromising of these keys through phishing or other targeted attacks can create vulnerabilities, potentially giving an attacker full control over a bridge.
**V13: Race Condition Attacks** A vulnerability is created when two or more processes access the same data, such as when withdrawals and deposits are processed without proper synchronization, and conflicting transactions can occur.
**V14: Unsafe External Call Exploits** When a smart contract accesses some external call to third-party contracts or off-chain contracts, these new calls create vulnerability in their own code security that can be exploited.
**V15: Forged Account Attacks** Occurs when an account or address is deliberately created to impersonate the identity of another party. This can lead to the redirection of tokens in a bridge structure if a fake account is validated.
**V16: Malicious Event Log Manipulation** Key actions are logged by smart contracts, typically off-chain. Falsification of the event log can lead to relaying of some fake events that have not happened. This results in tricking the chain into a false state.
**V17: Denial of Service Attacks** Bridge contracts can be flooded with transactions and requests sent by an attacker, overwhelming the bridge protocol and slowing processes for all transactions on the bridge. This could potentially lead to the complete halting of a bridge protocol.
**V18: Arithmetic Accuracy Deviation** Exact calculations are required to maintain consistency across chains. Tiny errors in integer overflow or underflow can cause precision errors, leading to inconsistency between chain states.
**Function Visibility and Modifier Use.**

# F DESCRIPTIONS OF HACKS AND ATTACKS ON BRIDGES

In one of the earliest and largest bridge hacks, attackers exploited a flaw in *Poly Network*'s cross-chain transaction handling contract, "literally tricking the project to hack itself". The *Poly Network bridge* connected multiple chains, including Ethereum, BSC, Polygon, and others. Its core contract (*EthCrossChainManage*) had an insecure design: it could make an external call to an arbitrary target contract with supplied data, without proper authorization checks. The attacker crafted a call payload that misled the manager contract into invoking *Poly*'s own storage contract (*EthCrossChainData*) and updating the keeper of funds as if the attacker were the owner. As a result, approximately $611 million worth of cryptocurrency was drained from *Poly Network* in a single attack, with the hacker subsequently returning the funds. The attack flow is illustrated in Figure 1.

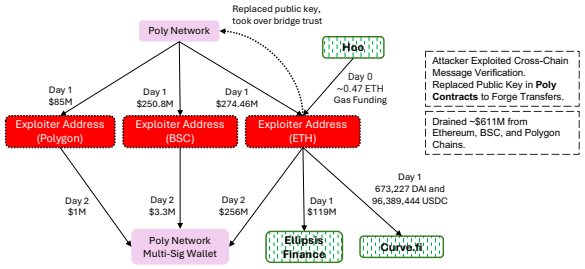


**Figure 1: Attack flow of the Poly Network exploit.**

The *Ronin bridge* to Ethereum was victim to a validator key compromise in an attack now attributed to North Korea's Lazarus Group. *Ronin* used a 5-of-9 multisig validation for bridge withdrawals. Attackers compromised five private keys through social engineering. Once in control, they issued two fraudulent transactions, draining 173,600 ETH and 25.5M USDC (worth approximately $624 million at the time) from the *Ronin bridge* in a single stroke. This event, the largest DeFi hack ever, was essentially a failure of the bridge's trust model: the off-chain validators were assumed honest, but the minimal quorum and centralized key management (a single entity controlled 4 of 9 validators) made it easy for an attacker to breach. The breach went unnoticed for six days until a user discovered it. Figure 2 summarizes the transaction flow of this exploit.

*Wormhole*'s Ethereum–Solana bridge was hacked for 120,000 WETH (worth $326 million) in a smart contract bug on the Solana side. *Wormhole* relies on a set of guardian nodes to attest cross-chain messages, generating a signed VAA (Verifiable Action Approval). On Solana, a *Wormhole* program should verify guardian signatures before minting new tokens. However, a critical mistake was introduced in an update: the Solana verification function was using an outdated system call for signature checks, effectively bypassing the actual verification. The attacker discovered that they could simply fabricate a data account that pretended to be a valid signature set, since the contract wasn't actually validating the signatures against the guardian keys. By exploiting this signature verification

**Table 4: Cross-Chain Bridge Implementations and Architecture**

| Bridge | Source Chain (Asset Custody & Locking) | Off-Chain Mechanism (Cross-Chain Validation) | Destination Chain (Asset Release & Minting) |
|---|---|---|---|
| **Allbridge Classic** | Smart Contracts – Assets locked in source-chain bridge contract[1] | Notaries/Relays – Small validator set observes lock and signs confirmations[2] | Smart Contracts – Target chain bridge contract verifies signature and mints tokens[3] |
| **deBridge** | Smart Contracts – Gateway contract escrows assets[4] | Notaries/Relays – Decentralized validator committee signs off-chain[5] | Smart Contracts – Destination contract verifies signatures and releases assets[6] |
| **Multichain** | Smart Contracts (MPC custody) – Deposited assets held by MPC-controlled vault[7] | Notaries/Relays – Multi-party computation (MPC) validators co-sign transactions[8] | Smart Contracts – Destination contract mints wrapped assets upon MPC authorization[9] |
| **Wormhole Bridge** | Smart Contracts – Core contract locks assets or emits transfer message[10] | Notaries/Relays – Guardian network signs verified action approvals (VAA)[11] | Smart Contracts – Destination chain contract verifies VAA and mints assets[12] |
| **Avalanche Bridge** | Validator Control – Ethereum escrow EOA with multi-sig custodian[13] | Notaries/Relays – Intel SGX enclave validators approve transactions[14] | Smart Contracts – Destination bridge contract mints assets upon SGX approval[15] |
| **Ronin Bridge** | Smart Contracts – Ethereum bridge contract locks assets[16] | Sidechain – Ronin validators (5 of 9 required) approve transfers[17] | Validator Control – Ronin validators execute mint/burn on sidechain[18] |
| **Wanchain Bridge** | Validator Control – Storeman group (25 nodes) holds escrow[19] | Notaries/Relays – Storeman nodes verify transactions using MPC[20] | Validator Control – Storeman group authorizes mint/release on target chain[21] |
| **Connext** | Smart Contracts – NXTP protocol escrow locks tokens[22] | Relays – Liquidity routers observe lock and relay signed messages[23] | Smart Contracts – Destination contract releases assets using router liquidity[24] |
| **Axelar** | Smart Contracts – Gateway contract on source chain locks assets[25] | Sidechain – Axelar validators reach consensus and use threshold cryptography[26] | Smart Contracts – Destination gateway releases tokens upon validator approval[27] |
| **Binance Bridge** | Validator Control – Binance custodial wallet holds assets[28] | Notaries/Relays – Binance-controlled relay system detects deposits[29] | Validator Control – Binance mints BTokens pegged to locked assets[30] |
| **Horizon Bridge** | Validator Control – 2-of-5 multi-signature custody on Ethereum[31] | Notaries/Relays – Federated validators approve transfers[32] | Validator Control – Multi-sig bridge mints tokens on Harmony[33] |
| **BTC Relay** | Validator Control – Bitcoin transactions are verified by BTC Relay's on-chain light client, but funds must be sent to an externally controlled address[34] | Light Client – BTC Relay runs a Merkle proof verification contract on Ethereum that allows Bitcoin transactions to be verified on-chain[35] | Smart Contracts – Ethereum contracts verify Bitcoin transactions and trigger asset issuance or validation[36] |
| **zkBridge** | Smart Contracts – Source chain tokens are locked in a bridge contract before proof generation[37] | Light Client – zk-SNARK proofs verify block headers across chains without external validators[38] | Smart Contracts – Destination chain contracts validate proofs and mint or release tokens[39] |
| **PeaceRelay** | Smart Contracts – Ethereum-based contracts escrow tokens and validate state transitions[40] | Light Client – Smart contracts maintain Merkle-Patricia proofs for cross-chain validation[41] | Smart Contracts – Contracts on Ethereum Classic or other chains verify proofs and mint assets[42] |
| **Rainbow Bridge** | Smart Contracts – NEAR and Ethereum smart contracts escrow tokens and initiate cross-chain transfers[43] | Light Client – Each chain hosts a light client verifying the other chain's consensus, eliminating external trust[44] | Smart Contracts – Contracts on NEAR or Ethereum validate and execute transactions upon light-client verification[45] |
| **Cosmos IBC** | Smart Contracts – The IBC module in Cosmos SDK escrows assets and generates proofs[46] | Light Client – Chains run light clients of each other, verifying block headers using the Tendermint consensus[47] | Smart Contracts – The receiving chain validates IBC messages and mints corresponding assets[48] |
| **Gravity Bridge** | Validator Control – Ethereum assets are locked in a Gravity.sol contract controlled by the Gravity Bridge validators[49] | Sidechain – Gravity Bridge validators observe Ethereum transactions and reach consensus on transfers[50] | Validator Control – Tokens are minted on Cosmos chains upon validator consensus[51] |
| **zkRelay** | Validator Control – Transactions from PoW chains like Bitcoin are observed, with no native smart contracts on the source chain[52] | Light Client – zk-SNARKs are used to verify large batches of Bitcoin headers efficiently[53] | Smart Contracts – Ethereum or other chains verify the zk-proof and execute transactions accordingly[54] |
| **Cactus** | Validator Control – A permissioned set of validators controls funds and relays messages[55] | Notaries/Relays – Cross-chain transactions are notarized by trusted relayers in enterprise settings[56] | Validator Control – The target blockchain executes transactions based on validator approvals[57] |
| **Celer cBridge** | Smart Contracts – Liquidity pools and lock-mint contracts facilitate asset transfers[58] | Sidechain/Relayer Network – Celer's State Guardian Network (SGN) validates transfers via PoS consensus[59] | Smart Contracts – cBridge smart contracts process releases based on SGN validators' signed messages[60] |
| **Orbit Bridge** | Validator Control – A federation of Orbit Chain validators governs asset custody[61] | Sidechain – Orbit Chain runs a dedicated blockchain that validates cross-chain transactions[62] | Validator Control – Assets are minted or unlocked on the target chain by Orbit validators[63] |

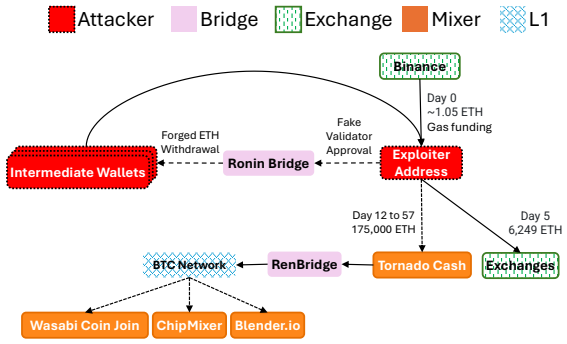


**Figure 2: Attack flow of the *Ronin* bridge exploit.**

bypass, the attacker called *Wormhole*'s complete_wrapped routine on Solana to mint 120k wrapped ETH for themselves with no real backing. The exploit structure is depicted in Figure 3.

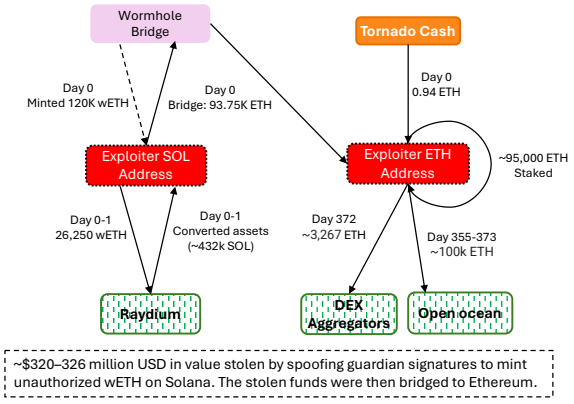


**Figure 3: Attack flow of the *Wormhole* bridge exploit.**

*Qubit Finance*'s bridge between Ethereum and BSC was hit for $80 million in an attack that abused improper input validation. *Qubit*'s contract allowed users to deposit an asset on Ethereum and mint a pegged version on BSC. The hack targeted the BSC side's deposit function: due to a logic error in a single overlooked condition, the bridge's smart contract did not verify that a call to transfer ETH actually happened when a deposit message was processed. In simpler terms, an attacker invoked the bridge deposit with zero ETH but bypassed the intended failure check, tricking the contract into believing collateral was provided. This exploit allowed the attacker to mint limitless xETH (the bridged ETH on BSC) without any real ETH locked on Ethereum. The exploit structure is depicted in Figure 4.

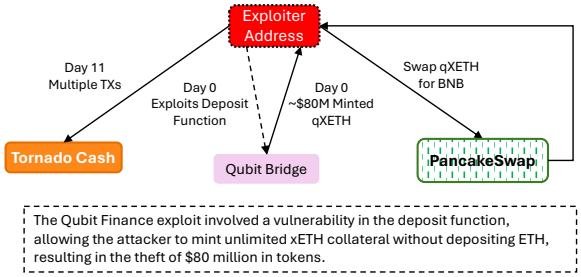


**Figure 4: Attack flow of the *Qubit* bridge exploit.**

The *Nomad* bridge (connecting Ethereum with Moonbeam and other chains) suffered a $190 million loss in one of the most chaotic exploits to date. *Nomad* was an optimistic messaging bridge, meaning it had a system of bonded updaters and a fraud challenge window. Unfortunately, after a routine smart contract upgrade, a critical initialization bug was introduced: the *Nomad* Replica contract on Moonbeam was set with a trusted root of 0x00…00 (zero) by mistake, which immediately enabled every message to be accepted as valid. Attackers quickly realized this and began submitting bogus withdrawal messages to drain tokens. The incident became a free-for-all: once the method was public, dozens of copycats (some white-hat, some black-hat) jumped in to also withdraw funds by replaying the attacker's transaction call data. The drain pattern across wallets is illustrated in Figure 5.

Harmony's *Horizon* bridge (connecting Harmony's chain with Ethereum) was hacked for about $100 million in assets, through a straightforward attack on its 2-of-5 multisig validators. Similar to *Ronin*, the attackers somehow obtained the private keys for at least two of the five signer addresses that controlled the bridge. Figure 6 shows the flow of compromised validator-controlled funds.

An attacker exploited the Binance Smart Chain's main bridge (between BSC and its Beacon Chain) and managed to mint 2 million BNB (≈ $586 million). The root cause was a bug in the light-client based proof verification on the BSC side. The bridge used an internal light client (IAVL tree-based) to verify bridge messages. The attacker created a fake proof for a past block that the BSC light

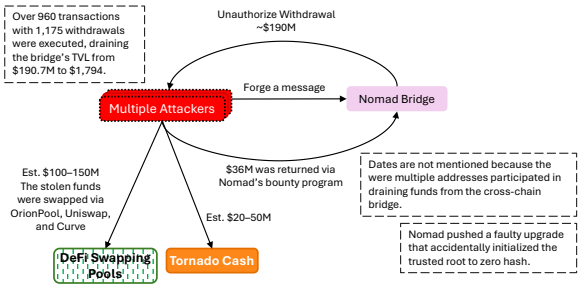


**Figure 5: Attack flow of the *Nomad* bridge exploit.**

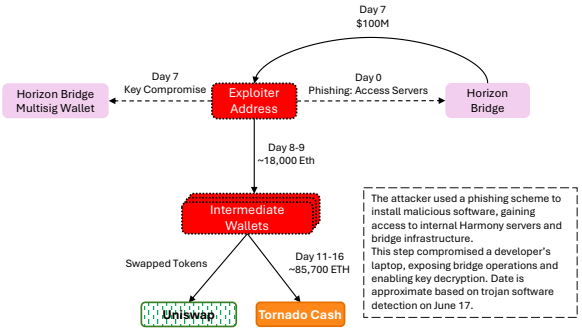


**Figure 6: Attack flow of the *Horizon* bridge exploit.**

client would accept as valid, bypassing the normal transaction inclusion checks. By forging this cryptographic proof, the attacker was able to inject a malicious block update that instructed the bridge to mint new BNB to their address. It was a complex exploit at the cryptography/verification layer, essentially an attack on the consensus verification between the two Binance chains. In response, Binance halted the entire BSC chain for 8 hours to prevent the attacker from moving more funds. A large portion of the illicit BNB never left Binance-controlled wallets, limiting the realized theft to around $100M. This incident revealed that even a seemingly trustless bridge (a light-client-based one) can have implementation flaws in its verification logic. The exploit structure is depicted in Figure 7.

In mid-2023, the *Multichain* bridge (formerly Anyswap), which linked over a dozen EVM chains, experienced a sudden breach resulting in approximately $126 million in assets withdrawn. Subsequent investigation suggested a compromise of the project's private keys or server infrastructure. It came to light that all of *Multichain*'s critical MPC key shares were under the control of its CEO, who had been detained by authorities in China. The attackers (or insiders) managed to use these keys to authorize massive transfers from the bridge's liquidity pools on multiple chains. Essentially, this was an insider compromise. The fallout also disrupted many linked chains' assets and led to the project's collapse. The fund outflow pattern across chains is shown in Figure 8.

Following Ethereum's transition from proof-of-work to proof-of-stake, the OmniBridge was exploited in September 2022 using a replay attack. Initially, the attacker sent 200 WETH using the OmniBridge on the Ethereum PoS chain. Since transaction format was

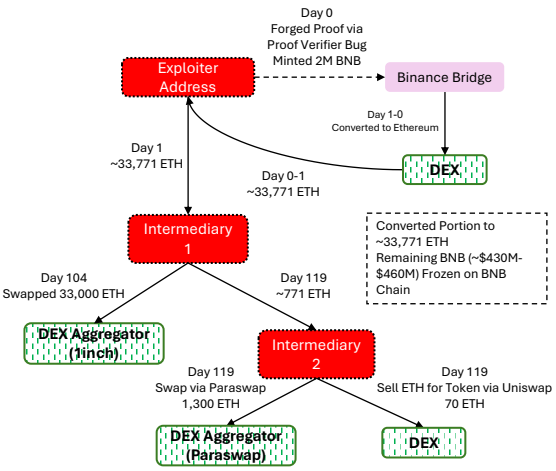


**Figure 7: Attack flow of the Binance exploit.**

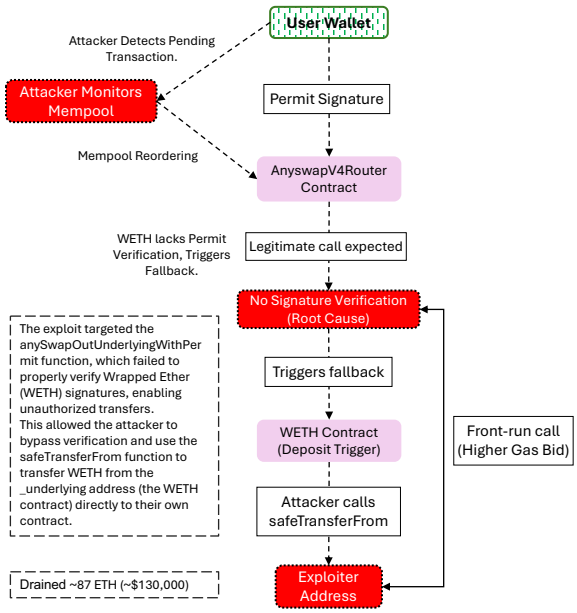


**Figure 8: Attack flow of the *Multichain* exploit.**

identical did not have chain ID verification, the attacker replayed the same transaction on the EthereumPoW fork. OmniBridge failed to distinguish between the two chains, therefore it processed the transaction and then released 200 ETHW to the attacker again. This exploit highlights the need for implementing strict chain ID validation during hard forks or network upgrades due to the possibility of having identical transaction histories across chains.

Smaller but instructive bridge attacks continued through 2024. For example, *Orbit Chain* (Jan 2024) lost $10M when 7 of its 10 bridge validators were compromised, another case of federated signer failure. An exploit in *ALEX Bridge* (May 2024) (connecting Stacks to other chains) also occurred, reportedly due to a logic bug

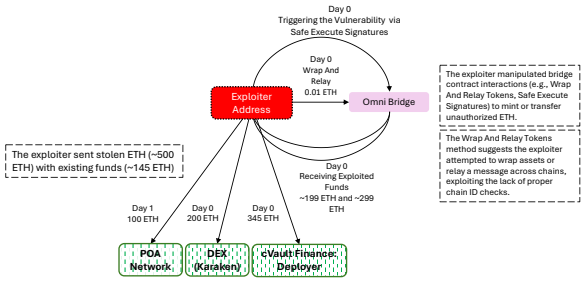


**Figure 9: Attack flow of the *Omni* bridge exploit.**

in its code. These ongoing incidents show that despite industry awareness, bridge security remains challenging.

## STATIC CODE TABLE COLUMN DESCRIPTIONS

Tables **??**, **??**, and **??** summarize features of bridge smart contracts extracted via static code analysis. Below, we define each column and its relevance.

- **Local vars**: Number of local (function-scoped) variables, indicating complexity of internal logic.
- **Inheritances**: Number of Solidity contracts inherited, reflecting code reuse or modularity.
- **Modifier Count**: Number of `modifier` constructs used to restrict access or enforce invariants.
- **RoleBased**: Indicates whether role-based access control mechanisms (e.g., `AccessControl`) are implemented.
- **Standard Libs**: Count of imported standard libraries such as `SafeMath` or `Ownable`.
- **LOC (Lines of Code)**: Number of non-comment, non-whitespace lines in the source code.
- **Total Lines**: Full line count including comments and whitespace.
- **Public / External / Internal / Private Funcs**: Number of functions by visibility. Public and external functions are externally callable and represent direct attack surfaces.
- **Global vars Declared**: Number of state variables declared at the contract level.
- **Ext Funcs**: Number of externally callable functions.
- **Low-level**: Number of low-level calls (`call`, `delegatecall`, `staticcall`).
- **Untrusted**: Instances where the target of a low-level call is not a hardcoded or trusted address.
- **Reentry Guard**: Presence of reentrancy protection (e.g., via the `nonReentrant` modifier).
- **Require / Assert**: Total number of `require` and `assert` statements used for validation and safety checks.
- **Custom Errors**: Count of Solidity custom error definitions used for gas-efficient error handling.
- **Checks/Fn**: Average number of `require` or `assert` checks per function, used as a proxy for defensive programming density.

## REFERENCES


[1] P. McCorry, C. Buckland, B. Yee, and D. Song, "Sok: Validating bridges as a scaling solution for blockchains," *Cryptology ePrint Archive*, 2021.

[2] Han'guk T'ongsin Hakhoe, IEEE Communications Society, Denshi Jōhō Tsūshin Gakkai (Japan). Tsūshin Sosaieti, Institute of Electrical, and E. Engineers, *ICTC 2018 : the 9th International Conference on ICT Convergence : "ICT Convergence Powered by Smart Intelligence" : October 17-19, 2018, Maison Glad Jeju, Jeju Island, Korea.* 2018.

[3] S.-S. Lee, A. Murashkin, M. Derka, and J. Gorzny, "SoK: Not Quite Water Under the Bridge: Review of Cross-Chain Bridge Hacks," in *2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–14, IEEE, 10 2023.

[4] G. Wang, Q. Wang, and S. Chen, "Exploring Blockchains Interoperability: A Systematic Survey," *ACM Computing Surveys*, vol. 55, pp. 1–38, 12 2023.

[5] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, "A Survey on Blockchain Interoperability: Past, Present, and Future Trends," 11 2022.

[6] A. Zamyatin, M. Al-Bassam, D. Zindros, E. Kokoris-Kogias, P. Moreno-Sanchez, A. Kiayias, and W. J. Knottenbelt, "SoK: Communication Across Distributed Ledgers," tech. rep., 2019.

[7] I. A. Qasse, M. A. Talib, and Q. Nasir, "Inter blockchain communication: A survey," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, 10 2019.

[8] S. K. Niranjan, REVA University, Institute of Electrical, E. E. B. Section, Institute of Electrical, and E. Engineers, *Proceedings of the International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE 2020) : October 9-10, 2020, Virtual Conference.* IEEE, 2020.

[9] S. Schulte, M. Sigwart, P. Frauenthaler, and M. Borkowski, "Towards Blockchain Interoperability," in *Business Process Management: Blockchain and Central and Eastern Europe Forum: BPM 2019 Blockchain and CEE Forum, Vienna, Austria, September 1–6, 2019, Proceedings 17*, pp. 3–10, 2019.

[10] T. Hardjono, A. Lipton, and A. Pentland, "Toward an Interoperability Architecture for Blockchain Autonomous Systems," *IEEE Transactions on Engineering Management*, vol. 67, pp. 1298–1309, 10 2020.

[11] P. Lafourcade and M. Lombard-Platet, "About Blockchain Interoperability," tech. rep., Université Clermont Auvergne, 2020.

[12] L. Duan, Y. Sun, W. Ni, S. Member, W. Ding, J. Liu, and W. Wang, "Attacks Against Cross-Chain Systems and Defense Approaches: A Contemporary Survey," *IEEE*, 2023.

[13] M. Borkowski, D. Mcdonald, C. Ritzer, and S. Schulte, "Towards Atomic Cross-Chain Token Transfers: State of the Art and Open Questions within TAST," tech. rep., TU Wien, 2018.

[14] G. Caldarelli, "Wrapping trust for interoperability: A preliminary study of wrapped tokens," 10 2022.

[15] M. Borkowski, P. Frauenthaler, M. Sigwart, T. Hukkinen, H. Hladk´y, and S. Schulte, "Cross-Blockchain Technologies: Review, State of the Art, and Outlook," tech. rep., TU Wien, 2019.

[16] M. Herlihy, B. Liskov, and L. Shrira, "Cross-Chain Deals and Adversarial Commerce," in *Proceedings of the VLDB Endowment*, vol. 13, pp. 100–113, 2019.

[17] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability," in *Network and Distributed System Security Symposium (NDSS)*, 2019.

[18] S. A. K. Thyagarajan, G. Malavolta, and P. Moreno-Sanchez, "Universal Atomic Swaps: Secure Exchange of Coins Across All Blockchains," in *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 1299–1316, 2022.

[19] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "FlyClient: Super-Light Clients for Cryptocurrencies," in *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 928–946, 2020.

[20] A. Kiayias, A. Miller, and D. Zindros, "Non-Interactive Proofs of Proof-of-Work," in *Financial Cryptography and Data Security (FC)*, vol. 12059 of *LNCS*, pp. 505–522, Springer, 2020.

[21] G. Scaffino, L. Aumayr, M. Bastankhah, Z. Avarikioti, and M. Maffei, "Alba: The Dawn of Scalable Bridges for Blockchains," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[22] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless Cross-Chain Bridges Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 3003–3017, ACM, 2022.

[23] A. Zamyatin, D. Harz, J. Lind, P. Panayiotou, A. Gervais, and W. Knottenbelt, "XCLAIM: Trustless, Interoperable, Cryptocurrency-Backed Assets," in *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 193–210, 2019.

[24] Z. Liao, Y. Nan, H. Liang, S. Hao, J. Zhai, J. Wu, and Z. Zheng, "SmartAxe: Detecting Cross-Chain Vulnerabilities in Bridge Smart Contracts via Fine-Grained Static Analysis," in *Proceedings of the 2024 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2024.

[25] "Near protocol." https://near.org/, 2025. Accessed: 2025-05-21. Official website of NEAR Protocol, a scalable, developer-friendly Layer 1 blockchain platform utilizing sharding (Nightshade) and a unique consensus mechanism (Doomslug)

**Table 5: Blockhain Bridge Exploits**

| Exploit Address | Attack Name | Attack Date | Bridge Name | Bridge Address |
|---|---|---|---|---|
| 0xc8a...963 | Poly Network Trusted State Root Exploit 1 | 2021-08-10 | Poly Network EthCrossChainManager | 0x144...88c |
| 0x5dc...214 | Poly Network Trusted State Root Exploit 2 | 2021-08-10 | Poly Network EthCrossChainManager | 0x144...88c |
| 0x123...678 | Multichain Rush Attack 1 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x9d5...b68 | Multichain Rush Attack 2 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0xefe...c88 | Multichain Rush Attack 3 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x418...bb7 | Multichain Rush Attack 4 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x622...ba0 | Multichain Rush Attack 5 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x48b...537 | Multichain Rush Attack 6 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x027...cd8 | Multichain Rush Attack 7 | 2023-02-15 | Multichain Router V4 | 0x6b7...522 |
| 0x489...bec | Binance Bridge Proof Verifier Bug | 2022-10-06 | Binance Bridge Ethereum Contract | 0x69F...66D |
| 0x82f...677 | Omni Bridge ChainID Vulnerability Exploit | 2022-09-18 | OmniBridge Multi-Token Mediator | 0x88a...671 |
| 0xb5c...90e | Nomad Trusted State Root Exploit 1 | 2022-08-01 | Nomad BridgeRouter | 0x88a...0a3 |
| 0x56D...4e3 | Nomad Trusted State Root Exploit 2 | 2022-08-01 | Nomad BridgeRouter | 0x88a...0a3 |
| 0xBF2...179 | Nomad Trusted State Root Exploit 3 | 2022-08-01 | Nomad BridgeRouter | 0x88a...0a3 |
| 0x0d0...d00 | Horizon Bridge Private Key Compromised | 2022-06-24 | Horizon Bridge | 0x2dc...857 |
| 0x098...f96 | Ronin Private Key Compromised (Social Engineering) | 2022-03-23 | Ronin Bridge Vault (V1) | 0x1A2...4F2 |
| 0x629...71a | Wormhole Account Spoofing | 2022-02-02 | Wormhole Portal Token Bridge | 0x3ee...585 |
| 0xd01...5c7 | Qubit Finance Deposit Function Exploit | 2022-01-28 | Qubit QBridge (Ethereum) | 0xD88...726 |
| 0x8c1...d62 | Thorchain Private Key Compromised (Phishing Attack) | 2021-07-24 | THORChain Bifrost ETH Router | 0xc14...2ce |


to support decentralized applications and smart contracts.

[26] "Aptos." https://aptosfoundation.org/, 2025. Accessed: 2025-05-21. Official website of Aptos, a high-performance Layer 1 blockchain developed by former Meta engineers. Aptos leverages the Move programming language and the Block-STM parallel execution engine to achieve high throughput and low latency, aiming to bring mainstream adoption to Web3.

[27] "Sui." https://sui.io/, 2025. Accessed: 2025-05-21. Official website of Sui, a high-performance Layer 1 blockchain designed for scalability and low latency. Sui utilizes the Move programming language and features an object-centric data model, enabling parallel transaction execution and instant settlement.

[28] "Cosmwasm." https://cosmwasm.com/, 2025. Accessed: 2025-05-21. Official website of CosmWasm, a WebAssembly-based smart contract platform designed for the Cosmos ecosystem, enabling secure, interoperable, and high-performance decentralized applications across multiple blockchains.

[29] "Base." https://base.org/, 2025. Accessed: 2025-05-21. Official website of Base, a secure, low-cost, builder-friendly Ethereum Layer 2 (L2) blockchain incubated by Coinbase. Base is designed to bring the next billion users onchain by offering fast transactions and low fees. It is built on the Optimism Superchain and aims to progressively decentralize over time.

[30] "Celo." https://celo.org/, 2025. Accessed: 2025-05-21. Official website of Celo, a mobile-first, carbon-negative blockchain platform that transitioned from an independent Layer 1 to an Ethereum Layer 2 solution. Celo focuses on real-world applications, offering low-cost transactions and supporting stablecoins like cUSD, cEUR, and cREAL.

[31] "Litecoin." https://litecoin.org/, 2025. Accessed: 2025-05-21. Official website of Litecoin, a peer-to-peer cryptocurrency enabling instant, near-zero cost payments to anyone in the world. Litecoin is an open-source, global payment network that is fully decentralized without any central authorities.

[32] "Blocknet." https://blocknet.co/, 2025. Accessed: 2025-05-21. Official website of Blocknet, a decentralized interoperability protocol that enables communication, exchange, and service delivery between different blockchains through a cross-chain architecture.

[33] "Colossusxt." https://www.colossusxt.io/, 2025. Accessed: 2025-05-21. Official website of ColossusXT, a community-oriented, energy-efficient cryptocurrency focused on decentralization, privacy, and real-world implementation.

[34] "Terra blockchain." https://www.terra.money/, 2025. Accessed: 2025-05-21. Official website of Terra, an open-source blockchain protocol that supports algorithmic stablecoins and decentralized applications. Terra experienced a significant collapse in May 2022, leading to the devaluation of its native tokens, LUNA and UST, and the subsequent launch of a new blockchain known as Terra 2.0.

[35] "The complete platform for onchain finance - wormhole." https://wormhole.com/institutions, 2025. Accessed: 2025-06-02.

[36] "Bridges | flow developer portal." https://developers.flow.com/ecosystem/bridges, 2025. Accessed: 2025-06-02.

[37] "Cardano." https://cardano.org/, 2025. Accessed: 2025-05-21. Official website of Cardano, a decentralized proof-of-stake blockchain platform built on peer-reviewed research, designed for secure and scalable smart contracts and decentralized applications.

[38] Cosmos Developer Portal, "What is IBC?." https://tutorials.cosmos.network/academy/3-ibc/1-what-is-ibc.html, 2024. Accessed: 2025-04-24.

[39] "Ibc/tao - clients - developer portal." https://tutorials.cosmos.network/academy/3-ibc/4-clients.html, 2025. Accessed: 2025-06-02.

[40] "Trustless interoperability between rollups: Landscape, constructions, and challenges." https://medium.com/1kxnetwork/trustless-interoperability-between-rollups-landscape-constructions-and-challenges-8ff195ea92cc, 2024. Accessed: 2025-06-02.

[41] L. T. Thibault, T. Sarry, and A. S. Hafid, "Blockchain scaling using rollups: A comprehensive survey," *IEEE Access*, vol. 10, pp. 93039–93054, 2022.

[42] B. Kiepuszewski, "Rollups are the most secure bridges." https://archive.devcon.org/devcon-6/rollups-are-the-most-secure-bridges/, 2022. Devcon 6, Bogota, October 13, 2022.

[43] "Axelar network." https://axelar.network/. Accessed: 2025-04-24.

[44] "Wormhole." https://wormhole.com/. Accessed: 2025-04-24.

[45] LayerZero Foundation, "Layerzero protocol." https://docs.layerzero.network/v2/developers/solana/getting-started, 2025. Accessed: 2025-04-24.

[46] "Multichain." https://multichain.org/. Accessed: 2025-04-24.

[47] "Celer cbridge." https://cbridge.celer.network/. Accessed: 2025-04-24.

[48] "Bnb chain." https://www.bnbchain.org/, 2025. Accessed: 2025-05-21. Official website of BNB Chain, a decentralized blockchain ecosystem supporting smart contracts and decentralized applications (dApps).

[49] "Bridging the gap: Ethereum-binance smart chain bridge (eth-bep)." https://www.binance.com/en/square/post/891550, 2023. Accessed: 2025-06-02.

[50] "Osmosis." https://osmosis.zone/, 2025. Accessed: 2025-05-21. Official website of Osmosis, a decentralized exchange (DEX) and Layer 1 blockchain in the Cosmos ecosystem, enabling cross-chain asset swaps and liquidity provision through the Inter-Blockchain Communication (IBC) protocol.

[51] "Cronos." https://cronos.org/, 2025. Accessed: 2025-05-21. Official website of Cronos, an open-source, EVM-compatible Layer 1 blockchain developed by Crypto.com. Built on the Cosmos SDK and powered by Ethermint, Cronos supports the Inter-Blockchain Communication (IBC) protocol, enabling interoperability with Ethereum and Cosmos ecosystems. It aims to scale decentralized applications (dApps) in DeFi, NFTs, and the Metaverse, offering low transaction fees and fast finality.

[52] "Gravity bridge." https://www.gravitybridge.net/, 2025. Accessed: 2025-05-21. Official website of Gravity Bridge, a decentralized, neutral Cosmos SDK-based

blockchain facilitating trustless interoperability between Ethereum and Cosmos ecosystems through the Inter-Blockchain Communication (IBC) protocol.

[53] "Exploring cross-chain solutions: How to bridge into the cosmos ecosystem." https://www.axelar.network/blog/cosmos-bridge-explained, 2023. Accessed: 2025-06-02.

[54] "Fantom." https://fantom.foundation/, 2025. Accessed: 2025-05-21. Official website of Fantom, a high-performance, scalable, and secure smart-contract platform designed for decentralized applications (dApps) and digital assets.

[55] deBridge Foundation, "debridge finance." https://debridge.finance, 2025. Accessed: 2025-04-24.

[56] "Polkadot." https://polkadot.com/, 2025. Accessed: 2025-05-21. Official website of Polkadot, a scalable, interoperable, and secure multi-chain blockchain platform designed to connect various blockchains into a unified network.

[57] Polkadot Documentation, "Introduction to xcm (cross-consensus messaging)." https://docs.polkadot.network/develop/interoperability/intro-to-xcm/, 2025. Accessed: 2025-04-24.